Technical Architecture Report: Reinforcement Learning for 6×6 Checkers

1 System Architecture Overview

This project implements a complete reinforcement learning (RL) pipeline for playing 6×6 checkers using a custom Gymnasium environment, a tabular Q-learning agent, and a curriculum-based training loop.

Core Components

Environment (`checkers_env.py`) The `Checkers6x6Env` class exposes a Gymnasium-compatible interface:

- **Observation space**: a dictionary

$$\text{obs} = \{\texttt{"board"} \in \mathbb{Z}^{6\times 6},\ \texttt{"current_player"} \in \{0,1\}\}$$

  where the board encodes:

$$0 = \text{empty},\ 1 = \text{P0 man},\ 2 = \text{P1 man},\ 3 = \text{P0 king},\ 4 = \text{P1 king}.$$

- **Action space**: MultiDiscrete($[6,6,6,6]$) representing $(\text{sr}, \text{sc}, \text{er}, \text{ec})$, i.e. start row/column and end row/column.
- **Game logic**: move generation, forced captures, promotions, multi-jumps, draw rules and rewards.

Q-Learning Agent (`q_agent.py`) The `QLearningAgent` is a tabular Q-learner:

- Uses a dictionary-based Q-table $Q(s,a)$ keyed by $(\text{state}, \text{action})$.
- Canonicalizes all states to a *Player 0 perspective*.
- Employs state-dependent ϵ-greedy exploration and a dynamic, per-(s,a) learning rate.
- Performs a backward-pass update over whole episodes with legal-action-aware bootstrapping.

Training Pipeline (`train.py`) The training pipeline:

- Instantiates the environment and agent.
- Runs episodes via `run_episode`, which:
  - Allows the agent to play as either Player 0 or Player 1.
  - Handles canonical state conversion and action flipping when the agent is Player 1.
  - Collects transitions in an `episode_memory`.

  - Invokes `backward_pass_update` after each episode.

- Implements a three-phase curriculum over opponent types (Random, Heuristic, Self-play).
- Maintains a historical opponent pool of Q-table snapshots for self-play stability.
- Periodically evaluates the agent with pure exploitation and no learning, saving `q_table.pkl` and `training_stats.npz` for analysis.

Supporting modules such as `play.py`, `heuristic_agent.py`, and `plots.py` enable evaluation, baselines, and visualization.

2 Environment Mechanics (`checkers_env.py`)

2.1 Board and Turn Representation

The environment maintains:

- A 6×6 integer board board$[r, c]$ with encoding

$$0 = \text{empty},\ 1 = \text{P0 man},\ 2 = \text{P1 man},\ 3 = \text{P0 king},\ 4 = \text{P1 king}.$$

- A scalar `current_player` $\in \{0, 1\}$ indicating whose turn it is.
- Only dark squares $(r + c) \bmod 2 = 1$ are used for pieces.

Initialization places Player 1 pieces (code 2) in the top two rows and Player 0 pieces (code 1) in the bottom two rows, consistent with checkers conventions.

2.2 Move Generation and Forced Captures

Legal moves are generated by `_get_legal_moves(player)`, which returns a pair:

$$(\text{simple moves},\ \text{capture moves}).$$

For each piece owned by `player`:

- Allowed directions are computed using `_move_directions(player, is_king)`:
  - Men move diagonally "forward": Player 0 moves up ($dr = -1$), Player 1 moves down ($dr = +1$).
  - Kings move in all four diagonal directions.
- A *simple move* is a one-step diagonal move onto an empty dark square.
- A *capture* is a two-step diagonal move landing on an empty dark square while jumping over an opponent piece.

The public helper `get_legal_actions(player)` enforces *mandatory captures*:

$$\text{if any captures exist, return only captures; otherwise return simple moves.}$$

This implements the forced capture rule: when a capture is available, non-capturing moves are illegal.

In `step`, a proposed action $(\text{sr}, \text{sc}, \text{er}, \text{ec})$ is checked against the current player's legal moves:

- If captures exist, only capture moves are accepted.
- Otherwise, any simple move from the legal list is accepted.
- Illegal moves cause no state change, no additional penalty, and the same player keeps the turn, although the training pipeline is now structured so that the agent only ever selects legal moves.

2.3 Multi-Jump State Machine

Multi-jump sequences (chained captures by the same piece) are handled as follows:

1. After applying a move via `_apply_move`, the environment sets a flag indicating whether a capture occurred: `captured_piece` $\in \{\text{False}, \text{True}\}$.
2. It also tracks whether the moving piece was promoted to a king through `_last_move_promoted`.
3. If a capture occurred and the piece was *not* promoted, the environment checks whether further captures are available from the new square:
   - Temporarily marks that square as the active piece.
   - Calls `_get_legal_moves` for the current player.
   - Checks whether any capture moves exist for that piece.
4. A boolean `continue_turn` is derived:

$$\texttt{continue_turn} = \texttt{captured_piece} \wedge \neg\texttt{promoted} \wedge \texttt{more_captures} \wedge \neg\texttt{terminated}.$$

5. If `continue_turn` is `True`:
   - The environment keeps `current_player` unchanged.
   - `active_piece` is set to the new square; subsequent moves are conceptually restricted to this piece.
6. Otherwise, `active_piece` is cleared and `current_player` is flipped to the opponent (unless the game has terminated).

The training and evaluation loops detect multi-jump situations by observing that the `current_player` is unchanged after a move and respond by having the same agent move again, effectively forming a state machine:

$$\text{Agent move} \rightarrow \begin{cases} \text{Agent again (multi-jump)} & \text{if capture + more captures,} \\ \text{Opponent move} & \text{otherwise.} \end{cases}$$

2.4 Draw Conditions and No-Progress Rule

To prevent degenerate games, the environment uses two draw rules:

- **Maximum steps**: if `step_count` $\geq$ `max_steps` (default 200), the game is drawn (winner $= -1$).
- **No-progress rule**: a counter `no_progress_counter` increments on moves that neither capture an opponent piece nor move a regular man (only king shuffling). It is reset on any capture or man move. When it reaches `max_no_progress_steps` (40), the game is drawn.

3 The Reward Function

The reward function combines terminal outcomes with shaping terms:

3.1 Components

For a single environment step, the reward is:

$$r = r_{\text{capture}} + r_{\text{promotion}} + r_{\text{terminal}} + r_{\text{step}},$$

where:

- $r_{\text{terminal}} \in \{-1, 0, +1\}$:
  - $+1.0$ if the current player wins (opponent has no pieces or no legal moves),
  - -1.0 if the current player loses,
  - 0.0 for draws (by no-progress or move-limit).
- $r_{\text{capture}} = +0.1$ if a capture occurred, else 0.
- $r_{\text{promotion}} = +0.15$ if the move promotes a man to king, else 0.
- $r_{\text{step}} = -0.005$ is a small negative applied on every move to penalize stalling.

From the agent's perspective in the training loop, opponent rewards are negated, so a positive reward for the environment when the opponent moves becomes a negative reward for the agent, and vice versa.

3.2 Magnitude Rationale

The magnitudes are chosen to ensure:

- **Terminal dominance**: Win/loss signals (± 1.0) dominate shaping rewards, steering the agent to optimize for game outcome rather than purely local gains.
- **Informative shaping**: Capture and promotion rewards ($+0.1, +0.15$) provide intermediate feedback about good tactics (trading up and advancing), while remaining subdominant to terminal outcomes.
- **Anti-stalling**: The small step penalty (-0.005) discourages unproductive move cycles (especially king shuffling) and encourages progress toward terminating states.
- **Numerical stability**: Keeping rewards in $[-1, 1]$ helps keep Q-values in a moderate range, facilitating stable learning with the dynamic learning rates and discount factor $\gamma \approx 0.99$.

4 State Canonicalization Logic (`q_agent.py`)

4.1 Player-0 Perspective Normalization

The function `observation_to_state` maps environment observations to canonical states:

- If `current_player` $= 0$, the board is used directly.
- If `current_player` $= 1$, the board is flipped vertically and horizontally, and piece IDs are remapped:

  $$1 \leftrightarrow 2, \quad 3 \leftrightarrow 4.$$

  This transforms "Player 1 to move" positions into strategically equivalent "Player 0 to move" positions, with the moving side's pieces always coded as 1/3 and the opponent as 2/4.

- The representation then discards all light squares and retains only the 18 dark squares where $(r + c) \bmod 2 = 1$.

The resulting state is a fixed-length 18-tuple:

$$s = (x_1, x_2, \ldots, x_{18}), \quad x_i \in \{0, 1, 2, 3, 4\}.$$

The `current_player` field is *not* included separately, as it is implicit in the canonicalization. This greatly reduces the effective state space and ensures that a single Q-table can be used for both players.

4.2 Action Symmetry

When the agent is assigned as Player 1 in training and evaluation:

- Legal actions are derived from the environment in environment coordinates.
- These are mapped into canonical coordinates via a symmetric flip

  $$(\mathrm{sr}, \mathrm{sc}, \mathrm{er}, \mathrm{ec}) \mapsto (5 - \mathrm{sr}, 5 - \mathrm{sc}, 5 - \mathrm{er}, 5 - \mathrm{ec}),$$

  before being passed to the Q-learning policy.
- After selecting an action in canonical space, it is flipped back to environment coordinates and executed.

Together, state canonicalization and action flipping ensure the agent learns a *color-agnostic* policy stored in a single Q-table.

5 Q-Learning Implementation

5.1 Tabular Q-Table

The Q-function is represented as a Python dictionary:

$$Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R},$$

with keys (s, a) where:

- s is the canonical 18-tuple state.
- $a = (\mathrm{sr}, \mathrm{sc}, \mathrm{er}, \mathrm{ec})$ is the action in canonical coordinates.

All $6^4 = 1296$ potential actions are precomputed from the action space, but policies and updates only operate over *legal* actions provided by the environment.

5.2 State-Dependent Epsilon-Greedy Exploration

The agent uses a state-dependent exploration rate:

$$\epsilon(s) = \frac{N_0}{N_0 + N(s)},$$

where:

- $N_0 > 0$ is a constant (set to 100 here).
- $N(s)$ is the count of how many times the agent has acted from state s.

The policy:

- Maintains a `state_visit_counts` dictionary.
- On each visit to s, increments $N(s)$ and computes $\epsilon(s)$.
- With probability $\epsilon(s)$, chooses a random action from the current list of legal actions.
- With probability $1 - \epsilon(s)$, selects the action a that maximizes $Q(s, a)$ over legal actions.

This contrasts with a global-decay ϵ, and has the advantages that:

- Rarely visited states remain exploratory even late in training.
- Frequently encountered states become increasingly exploitative.

5.3 Dynamic Learning Rate in Backward Pass

The backward-pass update processes the episode memory in reverse:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(s, a) \left[r + \gamma \max_{a' \in \mathcal{A}_{\text{legal}}(s')} Q(s', a') - Q(s, a) \right],$$

where:

- r is the immediate reward for the agent for the transition.
- s' is the next state (or None if terminal).
- γ is the discount factor (e.g. $\gamma = 0.99$).
- The max is taken only over *legal* actions in s'.

The learning rate is state–action specific:

$$\alpha(s, a) = \max \left(0.05, \ \frac{1}{\sqrt{N(s, a) + 1}} \right),$$

where $N(s, a)$ is the number of times the pair has been updated. This schedule:

- Starts with relatively large updates for new pairs, enabling fast learning.
- Decays as $1/\sqrt{N}$ to stabilize learning as more evidence accumulates.
- Never falls below 0.05, so the agent continues adapting even late in training, which is particularly important under self-play and changing opponents.

5.4 Legal-Action-Aware Targets

The target uses:

$$\max_{a' \in \mathcal{A}_{\text{legal}}(s')} Q(s', a'),$$

where $\mathcal{A}_{\text{legal}}(s')$ is the set of legal actions in the next state. If s' is terminal or there are no legal actions, the max term is taken as 0, reducing to a Monte Carlo-style return:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(s, a) \, [r - Q(s, a)].$$

Restricting the max to legal actions avoids overestimating Q-values based on impossible moves.

6 Curriculum Training Strategy (`train.py`)

6.1 Opponent Curriculum Phases

The training loop uses an auto-curriculum governed by a phase variable `current_curriculum_phase` $\in \{0, 1, 2\}$:

- **Phase 0 (Early)**:
  - Opponent types and probabilities:

$$P(\text{random}) = 0.8, \quad P(\text{heuristic}) = 0.2, \quad P(\text{self_play}) = 0.0.$$

  - The agent primarily learns basic tactics against random play, with occasional exposure to heuristic opponents.
- **Phase 1 (Intermediate)**:
  - Opponent probabilities:

$$P(\text{random}) = 0.1, \quad P(\text{heuristic}) = 0.8, \quad P(\text{self_play}) = 0.1.$$

  - Training focuses on beating the stronger heuristic baseline, while starting to encounter self-play.
- **Phase 2 (Advanced)**:
  - Opponent probabilities:

$$P(\text{random}) = 0.0, \quad P(\text{heuristic}) = 0.1, \quad P(\text{self_play}) = 0.9.$$

  - The agent predominantly plays against historical snapshots of itself, refining advanced strategies.

Phase transitions are performance-based. Every 1000 episodes, the agent is evaluated against the current opponent type using pure exploitation ($\epsilon = 0$) without learning. If the combined evaluation win rate exceeds 80%, the curriculum advances to the next phase (up to Phase 2).

6.2 Historical Opponent Pool and Catastrophic Forgetting

Self-play can induce catastrophic forgetting because the opponent distribution shifts as the agent improves. To stabilize learning:

- Every 5000 episodes, the current Q-table is deep-copied into a FIFO pool.
- The pool holds at most five snapshots; older ones are discarded when capacity is exceeded.
- When `opponent_type == "self_play"`, a random Q-table snapshot from the pool is used as the opponent's policy (acting greedily).

Advantages:

- Exposes the agent to a diverse set of "frozen" opponents drawn from its own training history.
- Encourages strategies that are robust across different versions of itself rather than only the current one.
- Mitigates non-stationarity in the opponent distribution, reducing catastrophic forgetting.

6.3 Symmetric Training as Both Colors

In each episode, the agent's color `agent_player_id` is randomly chosen as 0 or 1. This, combined with:

- state canonicalization to Player 0 perspective, and
- action flipping for Player 1,

ensures the single Q-table learns a policy effective for both sides.

7 Heuristic Agent Logic

The `PriorityHeuristicAgent` in `heuristic_agent.py` implements a rule-based policy with the following priority order:

1. **Forced capture**: If any capturing moves exist (identified by moves with row distance $|\text{er} - \text{sr}| = 2$), choose uniformly among them. This respects the game rule and promotes aggressive piece trading.
2. **King promotion**: If no captures exist, consider moves that promote a man to a king:
   - For Player 0, moves that land on the top rank ($\text{er} = 0$) from a man.
   - For Player 1, moves that land on the bottom rank ($\text{er} = \text{BOARD_SIZE} - 1$).

   If any such moves exist, select randomly among them.
3. **Edge safety**: If no promotions exist, consider moves whose destination column is on the board edge ($\text{ec} = 0$ or $\text{BOARD_SIZE} - 1$), as edges are sometimes safer.
4. **Advance center**: If no edge-safe moves qualify, compute a center score based on Euclidean distance to the board center:

   $$\text{center_score}(m) = (BOARD_SIZE - 1) - \sqrt{(er - c_r)^2 + (ec - c_c)^2},$$

   where (c_r, c_c) is the center coordinate (2.5, 2.5 on a 6×6 board). Moves are sorted by descending score, and one of the highest-scoring moves is chosen.
5. **Random**: If no other heuristic provides a strong preference, choose a move uniformly at random from the legal set.

This rule-based agent is significantly stronger than pure randomness and provides a meaningful baseline for the learning agent to surpass.

8 Conclusion and Potential Enhancements

8.1 Effectiveness of the Current Architecture

The system combines:

- A faithful and well-structured environment for 6×6 checkers with enforced game rules, multi-jump handling, and draw conditions.
- A compact and symmetric state representation via Player 0 canonicalization and dark-square filtering (18 cells).
- A robust tabular Q-learning implementation with:

  - Dynamic, state-dependent exploration $\epsilon(s)$,
  - Dynamic, per-(s, a) learning rates $\alpha(s, a)$,
  - Legal-action masking in bootstrap targets.

- A curriculum that transitions from random opponents to a heuristic baseline and finally to self-play, combined with a historical opponent pool to mitigate catastrophic forgetting.

Experimental infrastructure in `play.py` and `plots.py` supports both qualitative and quantitative evaluation, producing win-rate curves, state-space growth plots, and performance distributions.

8.2 Transition to Deep Q-Networks (DQN)

To extend this architecture from tabular Q-learning to Deep Q-Networks:

- **State encoding**: Replace the 18-tuple with:
  - A fixed-length vector (e.g. one-hot or embedding per cell), or
  - A multi-channel tensor (e.g. $6\times6\times C$) where channels encode piece types and the current player.

  The same Player 0 canonicalization can remain in place to exploit symmetry.

- **Network architecture**: Use a small convolutional network or multi-layer perceptron to approximate $Q(s, a; \theta)$, where θ are network parameters.

- **Action output and masking**: Either:
  - Output a Q-value for each of the 1296 possible actions and mask illegal actions before computing the max, or
  - Factorize the action into multiple discrete heads (e.g. for sr, sc, er, ec) with legal-move filtering.

- **Training mechanics**: Introduce an experience replay buffer, a target network, and minimize the mean squared TD error:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right].$$

The existing curriculum, state canonicalization, and opponent pool concepts are directly reusable; only the Q-function representation and learning rule change.

8.3 Alternative Algorithms

Beyond Q-learning and DQN, several alternatives are viable:

- **SARSA (on-policy)**:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\left[r + \gamma Q(s', a') - Q(s, a)\right],$$

  where a' is the actual next action taken under the current policy. This can be integrated into the existing backward-pass framework by storing the next actions and targets.

- **SARSA(λ) with eligibility traces**:

$$e(s,a) \leftarrow \gamma\lambda e(s,a) + 1_{\{(s,a)=(s_t,a_t)\}},$$
$$Q(s,a) \leftarrow Q(s,a) + \alpha\,\delta_t\, e(s,a),$$

  where δ_t is the TD error at time t and $\lambda \in [0,1]$ controls the degree of multi-step credit assignment. This is a principled extension of the backward-pass idea already in use.

- **Model-based approaches** (e.g. approximate value or policy iteration) that use the environment as a simulator to estimate value functions or policies.

- **Actor–Critic methods** (e.g. A2C, PPO) with deep function approximation, where:
  - The actor outputs a probability distribution over legal actions (with masking).
  - The critic estimates state value $V(s)$.

Overall, the current design forms a clean and extensible baseline for studying RL in small-board checkers, and provides a natural springboard for exploring deeper function approximation and more advanced algorithms.